Mission 3: Multi-Head Attention, RoPE Vector Rotation and Grouped-Query KV-Cache Optimization

Solution documentation, Squadron Bengio

- Course: Conversational AI: History, Applications, Future (BTU Cottbus-Senftenberg, WS 2026/27)
- Problem set: ps03, Week 3 (Modern Transformer Architectures and Architectural Divergence)
- Squadron: Bengio (Moodle Group 11)
- Repository: https://github.com/fahadakram009/squadron-bengio-cai (folder `mission-03`)
- Date: 25 October 2026

1. Mission objective

In the Pleiades cluster AURA's Subsystem 03, the spatial attention matrix and KV-cache transceiver, overflows because it processes long sensor streams with naive, untiled multi-head attention: compute grows with the square of the context length and the key-value cache saturates the memory bus. The mission rebuilds the core of the modern Transformer from first principles in pure PyTorch and analyses its memory behaviour: scaled dot-product and multi-head self-attention with causal masking and a variance-preservation test, rotary positional embeddings with a proof and an empirical check of relative-position invariance, grouped-query attention with a stateful KV cache and a memory and latency benchmark, and a written audit of FlashAttention's IO complexity, KV-cache denial of service and the PagedAttention and MLA mitigations. Unlike Missions 1 and 2, every deliverable is a runnable script or a derivation; no LLM API is needed.

2. Crew and stations

Station	Officer	Deliverable	Status
Systems Architect (Lead Engineer)	Fahad Akram	01_MultiHead_Attention_Core_starter.py, README, requirements, logs	signed off
Communications and Protocol Officer	Madhurima Lakshmi Prabha	02_RoPE_Positional_Engine_starter.py, figures/rope_vs_alibi_decay.png	signed off
Data and Payload Specialist	Osaf Ahmad	03_GQA_and_KVCache_Benchmark_starter.py	signed off
Security and Ethics Sentinel	Janusz Manczak	04_Memory_Hierarchy_and_Safety_Audit.md	signed off

The role titles follow this week's briefing; each officer kept the station they held in Missions 1 and 2. The Architect's script imports the Protocol Officer's and the Payload Specialist's scripts at run time and verifies them against its own attention module, and the Sentinel's audit cites the Architect's measured benchmark as its empirical starting point.

3. Repository layout and how to run

File	Content
README.md	mission overview, run instructions, station integration table, sign-off checklist
ps03_briefing.md	the instructor's mission specification
01_MultiHead_Attention_Core_starter.py	Station 01: multi-head self-attention, variance study, benchmark, 27-check self-test
02_RoPE_Positional_Engine_starter.py	Station 02: RoPE engine, invariance proof and checks, ALiBi comparison, extrapolation report
03_GQA_and_KVCache_Benchmark_starter.py	Station 03: GQA layer, KV cache, MHA / GQA / MQA benchmark
04_Memory_Hierarchy_and_Safety_Audit.md	Station 04: FlashAttention IO derivation, DoS simulation, mitigations
figures/rope_vs_alibi_decay.png	Station 02 figure
logs/*.json	committed self-test and benchmark evidence from the Architect's script

File	Content
requirements.txt	PyTorch and NumPy; matplotlib optional for the figure

Running the scripts (CPU is sufficient; the instructor's starter file names are kept so the grader's checklist matches):

```
source .venv/bin/activate && pip install -r mission-03/requirements.txt
cd mission-03
python 01_MultiHead_Attention_Core_starter.py --selftest   # 27 checks, imports 02 and 03
python 01_MultiHead_Attention_Core_starter.py --bench      # O(N^2) scaling
python 02_RoPE_Positional_Engine_starter.py                # 25 tests, extrapolation, figure
python 03_GQA_and_KVCache_Benchmark_starter.py             # GQA self-test + benchmark
python 01_MultiHead_Attention_Core_starter.py --check-signoffs
```

4. Station 01: Multi-Head Self-Attention from first principles (Fahad Akram)

The module keeps the starter's public surface, `MultiHeadSelfAttention` with `forward(x, causal, attention_mask)` returning the output and the attention weights, and implements the four items of the directive without any high-level attention wrapper in the forward path.

4.1 Implementation

The attention formula softmax(QK^T / sqrt(d_k)) V is written out in a free function: matrix product, scale, boolean mask to minus infinity, softmax, product with V. Rows that are masked everywhere, which occur with left padding under a causal mask, are returned as zero weights instead of NaN. The module owns four bias-free linear maps W_Q, W_K, W_V and W_O of size d_model by d_model, so the parameter count is exactly four times d_model squared. Head splitting reshapes (batch, seq_len, d_model) to (batch, seq_len, heads, head_dim) and transposes to (batch, heads, seq_len, head_dim); the merge is the exact inverse. The causal mask is a strictly lower-triangular boolean matrix cached per sequence length and combined by logical AND with an optional (batch, seq_len) key-padding mask. Dropout applies to the attention weights in training mode only. A `rope` hook on `forward` takes a callable that rotates Q and K after the head split, which is the integration point for Station 02.

4.2 Variance preservation

With zero-mean, unit-variance inputs each product q_i k_i has variance one, so the dot product over d_k dimensions has variance d_k, and dividing by sqrt(d_k) restores unit variance. The script measures this for d_k in 16, 64, 256 and 1024 on 32 by 128 random matrices.

d_k	Var(q.k) unscaled	Var(q.k / sqrt(d_k))	softmax entropy unscaled	softmax entropy scaled	maximum
16	16.3	1.019	2.47 bits	6.30 bits	7.00 bits
64	64.0	1.000	0.96 bits	6.30 bits	7.00 bits
256	255.4	0.998	0.43 bits	6.30 bits	7.00 bits
1024	1024.0	1.000	0.21 bits	6.30 bits	7.00 bits

The entropy columns show why the scale matters. Without it the logits grow with sqrt(d_k), the softmax becomes nearly one-hot and its gradient p(1 − p) vanishes; with it the attention distribution keeps about 6.3 of the possible 7 bits regardless of d_k.

4.3 Verification

The self-test runs 27 checks: output and weight shapes, rows summing to one, exact zeros above the diagonal, no information flow from a perturbed future position to earlier outputs under the causal mask (and flow when the mask is off), zero weight on padded keys with finite outputs for fully padded rows, the split and merge round trip, equality between the vectorised module and an explicit loop over heads followed by concatenation, equality with PyTorch's `F.scaled_dot_product_attention` and with `nn.MultiheadAttention` after copying the weights (maximum differences 1.5e-7 and 0), finite non-zero gradients for all four projections and the input, dropout active in training and determinism in evaluation, the variance table above, and the two integration checks described in section 8. The benchmark mode times a forward pass for N from 128 to 2048 and reports the score-matrix size, which grows exactly four times per doubling of N (0.25 MB to 64 MB per batch element at four heads); the wall time ratio approaches four once the N by N product dominates the projections.

5. Station 02: Rotary Positional Embeddings engine (Madhurima Lakshmi Prabha)

The engine computes the frequencies theta_i = base^(−2(i−1)/d), builds the complex unit phasors cis(m theta_i) and applies RoPE as a complex multiplication of each coordinate pair, keeping the starter's `precompute_freqs_cis` and `apply_rotary_emb` names so the Architect's module can import them unchanged. A real-valued `apply_rope`, an explicit cos/sin rotation and a dense block-diagonal rotation matrix R_m are provided as alternative paths and checked against each other.

The relative-invariance argument is written out in the file: treating each pair as a complex number, the dot product of the rotated query and key is the real part of a sum weighted by exp(j (m − n) theta_i), so it depends on the positions only through m − n, and in matrix form R_m^T R_n = R_(n−m). Empirically the script verifies that the dot product at positions (10, 25) equals the one at (50, 65) to 9.5e-7 and equals the closed form g(q, k, −15); a sweep of 2000 random (q, k, m, n, delta) combinations with positions up to 16384 holds to 7.7e-12 in float64, and a causal attention map is invariant to a global shift of 1000 positions. Its self-test passes 25 checks, including orthogonality and composition of R_m, norm preservation, gradient flow, half-precision handling and correct behaviour with fewer key heads than query heads.

Two analyses go beyond the directive. The decay comparison plots Su et al.'s upper bound on the RoPE score against distance next to ALiBi's linear penalty with the standard per-head slopes 1/2 to 1/256, and reports mean attention distance for both; the committed figure is `figures/rope_vs_alibi_decay.png`. The extrapolation report asks what happens when a model trained on 4096 tokens runs at 16384: plain RoPE exposes 18 of 64 frequency pairs to angles never seen in training (overshoot four times), position interpolation keeps every angle in range at the cost of four times coarser local resolution, NTK-aware base scaling leaves the fast pairs unchanged and slows the slowest by the scale factor while 17 pairs still overshoot slightly, and ALiBi needs no rotation but has a strong recency bias.

6. Station 03: Grouped-Query Attention and KV cache (Osaf Ahmad)

`GQAAttention` is a causal self-attention layer with a configurable number of query heads and key-value heads, where the query count must be divisible by the key-value count; with equal counts it is ordinary multi-head attention, with one key-value head it is multi-query attention. Key and value heads are expanded to the query heads with `repeat_interleave` before the score product. The forward pass accepts a past key-value tuple and a `use_cache` flag so that a single new token attends to the whole cached prefix, which is the autoregressive decoding path.

`KVCache` is an append-only buffer holding keys and values of shape (batch, key-value heads, sequence, head_dim). Its `append` validates rank, shape, device and dtype compatibility and concatenates the new token slice; the docstring states plainly that concatenation copies the cache on every step and that preallocated or paged storage is preferable for serving, which connects to Station 04.

The benchmark runs 32 query heads with head dimension 16, batch one, at N in 1024, 4096, 16384 and 32768 for MHA (32 key-value heads), GQA 4:1 (8) and MQA 32:1 (1). Cache size and bandwidth are analytical, latency is measured as the median of five one-token decodes against a cache of that length.

N	MHA cache	GQA 4:1 cache	MQA cache	MHA traffic at 50 tok/s	GQA traffic	MHA latency	GQA latency	MQA latency
1024	4.00 MB	1.00 MB	0.12 MB	0.21 GB/s	0.05 GB/s	0.27 ms	0.25 ms	0.24 ms
4096	16.00 MB	4.00 MB	0.50 MB	0.84 GB/s	0.21 GB/s	0.98 ms	0.73 ms	0.73 ms
16384	64.00 MB	16.00 MB	2.00 MB	3.36 GB/s	0.84 GB/s	3.22 ms	2.92 ms	2.79 ms
32768	128.00 MB	32.00 MB	4.00 MB	6.71 GB/s	1.68 GB/s	6.04 ms	5.70 ms	5.64 ms

GQA cuts the cache and the per-token traffic by 75 percent and MQA by about 97 percent at every length. The measured latency gain on a CPU with float32 is modest, between 5 and 25 percent, because this reference implementation expands the key-value heads back to 32 before the product; the memory and bandwidth savings, which are what matter on an accelerator, are exact. The script's own self-test checks shapes, the causal forward pass, cache appends and that a cached one-token decode is consistent with the full forward pass.

7. Station 04: Memory hierarchy and safety audit (Janusz Manczak)

The audit opens with the accelerator memory hierarchy, on-chip SRAM at about 19 TB/s against off-chip HBM at about 3.35 TB/s on an H100, and shows why standard attention is memory bound: it materialises the N by N score and probability matrices in HBM, giving IO of order N d + N². The quadratic term is made concrete with the Architect's benchmark log and extrapolated: at N = 65536 one score matrix in half precision is 8.6 GB, and across 32 heads and 32 layers 8.8 TB, which no device holds. The FlashAttention derivation then tiles Q into blocks of B_r rows and K, V into blocks of B_c columns sized to the SRAM capacity M, keeps the running maximum and normaliser of the online softmax so the full matrix never exists, and arrives at IO of order N d + N² d² / M, a reduction by a factor of about M / d² in HBM traffic.

The denial-of-service section parameterises the KV cache as 4 L H_KV d_head N B bytes in half precision

and models the ship's mainframe as four H100 80 GB cards holding a 70-billion-parameter model, which leaves 160 GB for dynamic memory. At N = 65536 a single MHA stream needs 171.8 GB and crashes before a second user connects; GQA 8:1 needs 21.5 GB per stream and sustains seven concurrent streams; MLA needs 12.1 GB and sustains thirteen. A table gives the same footprint across 1024 to 65536 tokens for the three architectures.

The mitigation section explains how PagedAttention replaces one contiguous reservation per request, which wastes 60 to 80 percent of VRAM on unwritten padding and fragments the pool, with fixed-size blocks addressed through a page table, recovering most of that memory. Multi-Head Latent Attention is derived from the DeepSeek-V3 formulation: keys and values are down-projected to a latent vector of dimension 512 plus a 64-dimensional decoupled rotary key, and the up-projections are absorbed into the query and output weights so that only the latent is cached. A comparison matrix for an 80-layer model gives 2.6 MB per token for MHA, 328 KB for GQA 8:1, 41 KB for MQA and 92 KB for MLA, and the audit closes with recommendations: per-request context quotas, paged allocation, GQA now and MLA for later missions, and a formal failure-mode review of the attention stack.

8. Integration across the four stations

Interface	What is checked	Result
Station 01 imports Station 02	attention run with the RoPE hook at positions 0..15 and at 37..52 must give the same attention map; rotation must change the map compared to no positions	both pass, maximum difference 1.2e-7
Station 01 imports Station 03	GQAAttention with equal query and key-value heads, weights copied from MultiHeadSelfAttention, causal, must give the same output	passes, difference below 1e-5
Station 02 and Station 03	apply_rotary_emb must work with fewer key heads than query heads (GQA 4:1)	passes in the Station 02 self-test
Station 04 reads Station 01	the measured score-matrix growth from logs/20261022-114613-ps03-bench .json anchors the IO derivation	cited in section 2.2 of the audit

The Payload Specialist's layer names its constructor arguments and projections differently from the instructor's starter (`num_query_heads`, `q_proj`); the Architect's check accepts both namings, and importing teammate scripts registers them as proper modules so that dataclass-based code loads.

9. Verification and sign-off

Check	Result
Station 01 self-test	27 of 27 pass with all teammates' final files (log committed)
Station 02 self-test	25 of 25 pass; invariance check within float32 tolerance
Station 03 self-test and benchmark	pass; table reproduced above
Sign-off gate	all four officers signed
Scripts run with plain python <file>.py	yes, CPU-only PyTorch 2.14, Python 3.12
Repository visibility	public

10. Limitations and honest notes

- All timings were taken on a laptop CPU in float32. Absolute latencies and the modest GQA speed-up are not representative of GPU half-precision serving, where the kernel is bandwidth-bound and the 75 percent traffic reduction translates more directly into speed.
- The Architect's benchmark reports the materialised score-matrix size analytically and the forward time empirically; it does not measure HBM traffic, which only the Sentinel's derivation addresses.
- The DoS simulation in the audit is an arithmetic model of a hypothetical four-H100 node, not an executed

out-of-memory experiment.
- The Payload Specialist first uploaded his script into the `logs/` folder through the GitHub web interface; the Architect moved it to its intended path. The Sentinel's audit contains one link to a path on his own machine rather than a repository-relative path.
- matplotlib is not installed in the shared virtual environment; the figure in `figures/` was produced on the Protocol Officer's machine and the script skips plotting gracefully when the library is absent.

11. Links

- Squad repository: https://github.com/fahadakram009/squadron-bengio-cai
- Mission 3 folder: https://github.com/fahadakram009/squadron-bengio-cai/tree/main/mission-03
- Instructor repository: https://github.com/Unisvet/couse-c-ai
- Course site: https://dlearn-cai.web.app